De Bruijn Notation and Recursion in the Faust Compiler

Synthesis Note

Abstract. This note presents the use of de Bruijn notation in the Faust compiler to represent the recursive forms produced by the ~ operator. After a review of the classical notation in lambda calculus (section 1), we describe the original adaptation that Faust makes: a group binder system with slot projections, tailored to multi-output feedback loops (section 2). We then detail the algorithm for converting recursive boxes to de Bruijn form during the propagation phase, covering nested and mutually recursive cases (section 3). Degenerate recursive forms and their simplification are addressed in section 4. Finally, section 5 describes the conversion to symbolic form, its algorithm, and its role for the later compiler phases.

---

1. De Bruijn Notation in Lambda Calculus

1.1 The Variable Binding Problem

In classical lambda calculus, bound variables are named arbitrarily. The terms `λx. x` and `λy. y` denote the same function (they are *alpha-equivalent*), but their syntactic representations differ. This naming ambiguity raises two practical problems for automated processing:

- **Alpha-equivalence** cannot be checked by simple structural comparison: one must reason modulo renaming.
- **Capture-avoiding substitution** is fragile: naively substituting a term containing free variables under a binder can accidentally *capture* those variables, altering the meaning of the term.

1.2 De Bruijn's Solution (1972)

In his foundational paper [de Bruijn 1972], Nicolaas Govert de Bruijn proposed replacing bound variable names with natural numbers. Each number (a *de Bruijn index*) encodes the distance — measured in the number of binders traversed — between the variable occurrence and the binder that binds it. Names disappear entirely:

```
λx. x              →  λ. 1
λx. λy. x          →  λ. λ. 2
λx. λy. y          →  λ. λ. 1
λx. λy. λz. x z (y z)  →  λ. λ. λ. 3 1 (2 1)
```

The semantic principle is straightforward:

- 1 denotes the nearest binder;
- 2 denotes the next binder outward;
- and so on.

1.3 Fundamental Properties

This representation has two key properties:

1. **Alpha-equivalence becomes structural equality.** Two terms are alpha-equivalent if and only if their de Bruijn representations are identical. No renaming is needed.
2. **Substitution and lifting become purely structural operations.** When substituting a term under additional binders, the free variables of the substituted term must be incremented to account for the new binders in scope. This is the *shift* (or *lift*) operation, which is purely mechanical in de Bruijn notation.

1.4 Indices vs. Levels

There is a dual formulation:

- **Indices** count from the variable occurrence *upward* to its binder (relative addressing).
- **Levels** count from the outermost scope *downward* to the binder (absolute addressing).

The Faust compiler uses the index convention (counting from the reference toward the nearest enclosing binder).

1.5 Beyond Unary Lambda Calculus

Recent literature shows that de Bruijn notation is not limited to unary lambda calculus. Keuchel and Jeuring [2012] explicitly show that well-scoped de Bruijn representations can describe multiple binders, sequential scopes, and recursive scopes. The notation has also been extended to recursive types (μ-types), higher-order rewriting [Bonelli, Kesner, Rios 2000], and process calculi [Perera, Cheney 2017].

The general idea of using de Bruijn beyond pure lambda calculus is therefore not specific to Faust. What is specific is the *kind of structure* that Faust chooses to encode this way.

---

2. The Original Adaptation in the Faust Compiler

2.1 Context: the ~ Operator and Feedback Recursion

In Faust, source programs do not contain `DEBRUIJNREC` or `DEBRUIJNREF` nodes. Recursion is written at the box-language level, via the ~ (tilde) operator or, in the box API, via `boxRec`. The official Faust documentation highlights two important facts:

- the semantic phase translates the program into signals by *symbolic propagation*;
- recursion automatically inserts a one-sample delay to guarantee causality.

For example:

```
process = + ~ *(0.5);
```

describes a one-sample-delay feedback loop: the output is fed back, multiplied by 0.5, and added to the input.

2.2 The Two De Bruijn Nodes in Faust

The internal representation uses two main forms:

Node	Role
`DEBRUIJNREC(body)`	**Binder** for a recursive group. Analogous to λ or μ.
`DEBRUIJNREF(level)`	**Reference** to an enclosing binder. Level 1 = innermost.

2.3 Faust's Specificity: the Group Binder with Projections

Faust's adaptation differs from classical lambda calculus in one decisive way: **the binder does not bind a single variable, but a group of outputs.**

A recursive reference on its own (`DEBRUIJNREF(1)`) is not directly usable. It is almost always combined with a slot projection:

```
proj(i, DEBRUIJNREF(1))
```

And the central causal-feedback pattern is:

```
delay1(proj(i, DEBRUIJNREF(1)))
```

In other words, Faust does not use de Bruijn indices to name lambda-term variables, but to **identify the current recursive group within a multi-output signal graph**. Slot selection is deferred to `proj(i, ...)`.

This is, to our knowledge, the most interesting specificity of this intermediate representation. No Faust publication explicitly documents this connection with de Bruijn notation, although the reference C++ compiler uses the same encoding (`rec`/`ref` with de Bruijn levels).

2.4 Why Not Use Named Variables Directly?

Three practical reasons justify the choice of de Bruijn during the propagation phase:

1. **Structural sharing.** The term arena (`TreeArena`) interns nodes by structural identity. De Bruijn nodes produce deterministic shapes independent of naming context, maximizing sharing.
2. **Correct scoping by construction.** Nested ~ operators produce nested `DEBRUIJNREC` binders; inner references automatically point to the correct scope via their level number — no alpha-renaming pass is needed.
3. **Standard technique.** The C++ Faust compiler uses the same encoding, ensuring structural parity with the Rust port.

---

3. Converting Recursive Boxes to De Bruijn Notation

3.1 The Framework: Recursive Composition `A ~ B`

At the box algebra level, `A ~ B` constructs a recursive composition. If:

- `A : Li → Lo` (the main body)

- `B : Ri → Ro` (the feedback path)

then the composition is well-formed when `Ri ≤ Lo` and `Ro ≤ Li`, and the result has arity `(Li — Ro) → Lo`.

Intuition: - `B` reads some outputs of `A`; - `B` produces feedback signals that feed into some inputs of `A`; - the feedback is always delayed by one sample, so the cycle is causal.

3.2 The Propagation Algorithm Step by Step

When the propagator encounters a `FlatNodeKind::Rec(left, right)` node, it executes the following steps:

Step 1 — Arity check.

```
left  : Li → Lo
right : Ri → Ro
Require: Ri ≤ Lo  AND  Ro ≤ Li
```

Step 2 — Create feedback placeholders. For each of the `Ri` feedback channels, create a placeholder signal:

```
l0[i] = delay1(proj(i, DEBRUIJNREF(1)))    for i = 0..Ri−1
```

This means: "the i-th feedback input is the previous sample (`delay1`) of the i-th projection (`proj`) of the recursive group currently being defined (`DEBRUIJNREF(1)`)."

Step 3 — Propagate the feedback path.

```
l1 = propagate(right, l0)
```

Step 4 — Build the full input vector.

```
rec_inputs = l1 ++ lift(external_inputs)
```

The body `left` receives first the `Ro` feedback signals, then the `Li — Ro` external inputs, **lifted** by one de Bruijn level.

Step 5 — Lift the slot environment.

```
slot_env' = { k → liftn(v, 1) | (k, v) ∈ slot_env }
```

Any slot-environment values containing `DEBRUIJNREF` nodes must be lifted to avoid capture by the new inner binder.

Step 6 — Propagate the body.

```
l2 = propagate(left, rec_inputs)    // using slot_env'
```

Step 7 — Wrap and project.

```
group = DEBRUIJNREC(list(l2[0], l2[1], ..., l2[Lo−1]))

output[i] =
  if aperture(l2[i]) > 0: proj(i, group)   // truly recursive
  else:                    l2[i]           // closed form, emitted directly
```

3.3 Simple Example: `+ ~ *(0.5)`

```
process = + ~ *(0.5);
```

The compiler first builds the feedback seed:

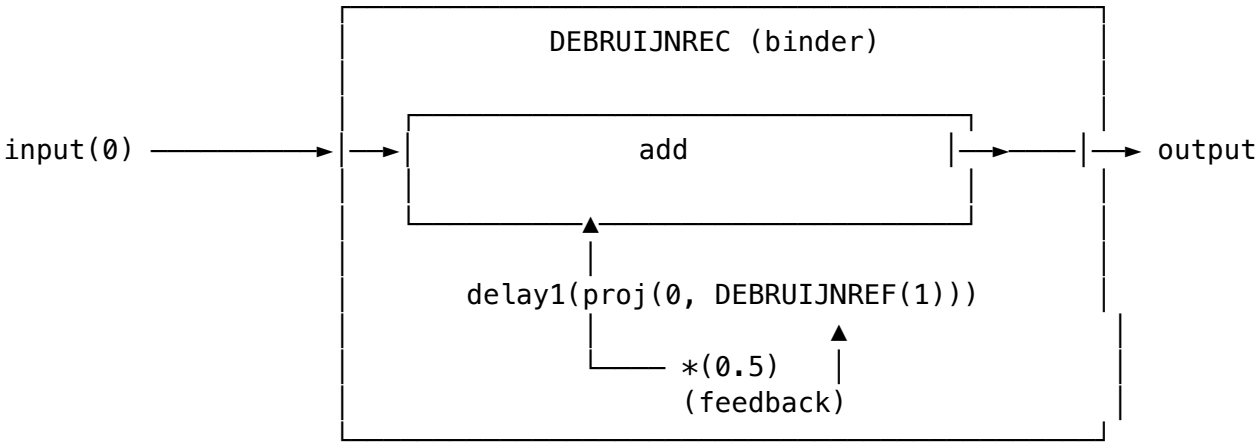
```
delay1(proj(0, DEBRUIJNREF(1)))
```

The feedback path `*(0.5)` transforms it, and the body + builds:

```
body0 = add(
  delay1(proj(0, DEBRUIJNREF(1))) * 0.5,
  input(0)
)

group = DEBRUIJNREC([body0])
out0  = proj(0, group)
```

Diagram:



3.4 Nested Recursion

Consider a form where one recursion is itself placed inside another:

```
inner = + ~ *(0.5);
process = inner ~ *(0.25);
```

The recursion defined by `inner` is itself placed under a new recursive binder. The consequence is standard de Bruijn behavior: every free reference coming from the outer scope must be **lifted by one level** so that it is not captured by the inner binder.

Schematically, one obtains a structure of the form:

```
DEBRUIJNREC(                    ← outer group
  ...
  DEBRUIJNREC(                  ← inner group
    pair(
      DEBRUIJNREF(1),           ← points to the inner group
```

```
      DEBRUIJNREF(2)            ← points to the outer group
    )
  )
  ...
)
```

The lifting algorithm (`liftn`):

```
liftn(node, threshold):
  if node = DEBRUIJNREF(level):
    return DEBRUIJNREF(level)       if level < threshold
    return DEBRUIJNREF(level + 1)   otherwise

  if node = DEBRUIJNREC(body):
    return DEBRUIJNREC(liftn(body, threshold + 1))

  otherwise:
    rebuild the node by applying liftn to each child
```

The `threshold + 1` is the key point. Descending under an inner recursive binder means that one additional level becomes locally bound. The criterion for "freeness" must therefore shift accordingly.

Concrete example. Suppose a value coming from an outer recursion already contains:

```
delay1(proj(0, DEBRUIJNREF(1)))
```

and this value is reused inside a new `Rec`. Without lifting, `DEBRUIJNREF(1)` would now be interpreted as "the new inner group," even though it originally meant "the already existing outer group."

After `liftn(..., 1)`:

```
delay1(proj(0, DEBRUIJNREF(2)))
```

Lexical meaning is preserved: - `DEBRUIJNREF(1)` now denotes the newly created inner group; - `DEBRUIJNREF(2)` continues to denote the old outer group.

Why Faust must lift in two places. In `propagate`, this operation is applied to two families of objects:

1. The **inputs** injected into the `left` body of the `Rec`;
2. The **`slot_env` values** (slot environment).

The second point is easy to miss but essential. A value produced by a box abstraction or a local definition may contain `DEBRUIJNREF` nodes coming from an outer loop. If it is reinjected without lifting into an inner loop, it will be silently captured by the wrong binder.

3.5 Multi-Output Recursion and Mutual Recursion

In Faust, **mutual recursion is a special case of multi-output recursion**, not a synonym. Both use the same mechanism: a single `DEBRUIJNREC` binder wrapping a vector of bodies.

```
import("stdfaust.lib");
feedback = si.bus(2) ~ (*(0.5), *(0.25));
process = feedback;
```

Multi-output recursion (each channel feeds itself) This example represents a recursive group with two bodies:

```
DEBRUIJNREC([
  body0 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5,
  body1 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25
])
```

Each channel depends on its own slot: `body0` projects slot 0, `body1` projects slot 1. This is multi-output recursion, but not mutual recursion in the strict sense.

Genuinely mutual recursion (signals cross) To obtain true mutual recursion, the signals must be crossed inside the recursive loop. The `ro.cross(2)` operator swaps two signals:

```
import("stdfaust.lib");
process = si.bus(2) ~ ((*(0.5), *(0.25)) : ro.cross(2));
```

The crossing makes each output depend on the *other* output:

```
DEBRUIJNREC([
  body0 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25,
  body1 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5
])
```

Semantically, the two signals are coupled:

```
output0[n] = 0.25 × output1[n−1]
output1[n] = 0.5  × output0[n−1]
```

The key point The binder is shared by the entire output vector, not one binder per output. Mutually recursive outputs are not represented by a different mechanism — they are a special case of a multi-output group with crossed projections. From the compiler's perspective, the only difference is *which projections appear in each body*.

---

4. Degenerate Recursive Forms

4.1 What Is a Degenerate Form?

A recursive form is called *degenerate* when the material recursive group still exists, but one or more of its outputs **no longer actually depend on feedback**. This happens when some branches of the feedback path:

- explicitly discard their recursive input;
- or become closed after propagation and simplification.

4.2 Representative Example

The classic case is a multi-channel bus where most channels ignore their feedback:

```
import("stdfaust.lib");

N = 8;
gain = hslider("gain", 0.5, 0.0, 0.99, 0.01);

process = si.bus(N) ~ (!, !, !, !, !, !, !, *(gain));
```

Here, channels 0 through 6 discard their recursive input via `!`; only channel 7 remains genuinely recursive. The real-world trigger for this problem was `re.zita_rev1_stereo(...)` (file `Birds.dsp`), an 8-delay-line algorithmic reverb whose feedback matrix produced exactly this shape after evaluation and propagation.

4.3 Detection via Aperture

The conceptual tool for detecting degenerate branches is the **aperture**, defined as the maximum free de Bruijn reference level in a subtree:

```
aperture(DEBRUIJNREF(k))      = k
aperture(DEBRUIJNREC(body))   = aperture(body) − 1
aperture(other node)          = max(aperture(children))
```

Interpretation: - `aperture > 0`: the branch is still open over the recursive group; - `aperture` $\leq$ `0`: the branch is closed at that boundary.

In `propagate`, a closed output is no longer emitted as `proj(i, group)` but directly as a raw expression.

4.4 The Shifted Projection Index Problem

Detection via aperture is not sufficient to eliminate all structural degeneracy. In the classic C++ pipeline, a more aggressive pass, `inlineDegenerateRecursions()`, can compact a group and keep only the genuinely recursive bodies. This creates a subtle problem: the logical projection index may remain the original one, while the physical arity of the group has shrunk:

```
before compaction: proj(7, SYMREC([b0, ..., b7]))
after compaction:  proj(7, SYMREC([b7]))
```

The group now has only one physical body, but the projection still says 7. For a backend that models recursive slots as a `Vec`, this is an out-of-bounds index.

4.5 Simplification in the Rust Port

In the current Rust port, the adopted simplification is narrower: in `signal_prepare`, every projection targeting a unary symbolic group is **canonicalized** to `proj(0, group)`:

```
before: SYMREC(W, [body_7])  with  proj(7, W)
after:  SYMREC(W, [body_7])  with  proj(0, W)
```

The stakes are not cosmetic. Simplifying degenerate forms is useful to:

- keep slot indices dense;
- stabilize typing and FIR generation;
- avoid “projection index out of bounds” errors downstream.

---

5. Converting De Bruijn Form to Symbolic Form

5.1 Why Convert?

De Bruijn form is excellent during propagation:

- no name generation is needed;
- scopes are correct by construction;
- structural sharing in the arena is maximized.

However, it is neither very readable nor very convenient for later passes. Mentally counting binder depths inside a shared signal DAG is tolerable for `propagate`, much less so for typing, FIR preparation, recursive-group lowering, and diagnostics.

Symbolic form therefore replaces:

De Bruijn form	Symbolic form
`DEBRUIJNREC(body)`	`SYMREC(var, body)`
`DEBRUIJNREF(level)`	`SYMREF(var)`

5.2 The Conversion Algorithm

The `de_bruijn_to_sym(...)` algorithm, shared between the historical C++ compiler and the Rust port, is conceptually:

```
convert(node):
  if node = DEBRUIJNREC(body):
    var = fresh("W")                                  // e.g., W0, W1, ...
    body1 = substitute(body, level=1, replacement=SYMREF(var))
    body2 = convert(body1)
    return SYMREC(var, body2)

  if node = DEBRUIJNREF(level):
    error: the converted root was open

  otherwise:
    recursively rebuild the node
```

The subtle point is the **substitution**:

```
substitute(node, level, repl):
  if aperture(node) < level:
    return node                          // no free reference at this level
```

```
  if node = DEBRUIJNREF(level):
    return repl                          // direct replacement
  if node = DEBRUIJNREC(body):
    return DEBRUIJNREC(substitute(body, level + 1, repl))
  otherwise:
    rebuild by applying substitute to children
```

When descending under a nested `DEBRUIJNREC`, the searched level becomes `level + 1`, exactly as in `liftn`. This is what makes correct conversion of nested trees possible.

5.3 Nested Example

Input:

```
DEBRUIJNREC(
  DEBRUIJNREC(
    pair(DEBRUIJNREF(1), DEBRUIJNREF(2))
  )
)
```

The conversion produces:

```
SYMREC(W0,
  SYMREC(W1,
    pair(SYMREF(W1), SYMREF(W0))
  )
)
```

The expected lexical logic is recovered: - `DEBRUIJNREF(1)` in the inner group $\rightarrow$ `SYMREF(W1)` (nearest binder); - `DEBRUIJNREF(2)` in the same group $\rightarrow$ `SYMREF(W0)` (outer binder).

5.4 Properties of the Conversion

The conversion is a **representation change**, not a structural normalization:

- binder nesting remains identical;
- the recursive body list remains identical;
- projection indices (`proj(i, ...)`) remain identical;
- only the recursion carrier changes, from positional references to named ones.

5.5 Use by Later Compiler Phases

In the Rust port, `prepare_signals_for_fir(...)` clones the entire output forest, applies `de_bruijn_to_sym(...)` to the complete list, then performs in order:

1. De Bruijn to symbolic conversion;
2. Canonicalization of degenerate unary projections;
3. Reduced type inference (Int/Real/Sound);
4. Signal promotion casts;
5. FIR preparation.

The FIR lowerer then expects groups of the form:

```
SYMREC(var, body_list)
SYMREF(var)
```

and no more raw `DEBRUIJNREC` / `DEBRUIJNREF` nodes. This choice brings three practical benefits:

- it makes the identity of the recursive group explicit;
- it allows `signal_fir` to directly decode the body list of a symbolic group;
- it establishes a clean pipeline boundary: after preparation, the backend no longer needs to reason in terms of lexical depths.

In summary, the conversion is not merely cosmetic. It is a representation change that separates **scope logic** (useful during propagation) from **backend consumption logic** (useful during lowering).

---

6. Conclusion

In the Faust compiler, de Bruijn notation plays a more concrete role than in many purely theoretical presentations: it serves as a working form for building feedback groups that are correct by construction, even in the presence of nesting, multi-output groups, and structural sharing.

The most important point is not simply "Faust uses de Bruijn," which would be too general, but rather:

- Faust treats recursion as a **group binder**, not as a single variable;
- recursive references are addressed first by **depth**, then by **slot projection**;
- degenerate forms impose a **canonicalization** discipline;
- the final conversion to `SYMREC` / `SYMREF` **isolates** downstream passes from scope complexity.

From this perspective, de Bruijn form in Faust is both classical in principle and highly specialized in compiler use.

References

External Sources

- N. G. de Bruijn, "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem," *Indagationes Mathematicae*, vol. 34, pp. 381-392, 1972. https://research.tue.nl/en/publications/lambda-calculus-notation-with-nameless-dummies-a-tool-for-automat-2/
- S. Keuchel, J. T. Jeuring, "Generic Conversions of Abstract Syntax Representations," WGP 2012. https://ics-archive.science.uu.nl/research/techreps/repo/CS-2012/2012-009.pdf
- E. Bonelli, D. Kesner, A. Rios, "A de Bruijn notation for higher-order rewriting," RTA 2000. https://doi.org/10.1007/10721975_5
- Y. Orlarey, D. Fober, S. Letz, "Syntactical and Semantical Aspects of Faust," *Soft Computing*, vol. 8, 2004. https://link.springer.com/article/10.1007/s00500-004-0388-1

- Y. Orlarey, S. Letz, D. Fober, R. Michon, “A New Intermediate Representation for Compiling and Optimizing Faust Code,” International Faust Conference, 2020. https://hal.science/hal-03124677
- Faust Documentation, “Using the box API.” https://faustdoc.grame.fr/tutorials/box-api/

Internal Repository Sources

- debruijn-recursion-faust-note-en.md — detailed note on de Bruijn and recursion in Faust.
- recursion-debruijn-lowering-en.md — internal design document on the lowering.
- flatnode-rec-to-signals-en.md — operational description of the `FlatNodeKind::Rec` to signals conversion.
- `crates/propagate/src/lib.rs` — propagation implementation.
- `crates/tlib/src/recursion.rs` — de Bruijn to symbolic conversion, lifting, aperture.
- `crates/transform/src/signal_prepare.rs` — degenerate unary recursion canonicalization.
- `tests/corpus/rep_71_degenerate_unary_recursion.dsp` — degenerate form regression.
- `tests/corpus/rep_79_multi_output_recursion.dsp` — multi-output recursion regression.
- `tests/corpus/rep_80_mutual_recursion_crossed.dsp` — mutual recursion regression.